CSCI 4100

Operating Systems and Architecture

Chapter 2: The Kernel Abstraction

Topics

- 2.0 – The Kernel Abstraction
- 2.1 – The Process Abstraction
- 2.2 – Dual-Mode Operation
- 2.3 – Types of Mode Transfer
- 2.4 – Implementing Safe Mode Transfer
- 2.5 – Putting It All Together: x86 Mode Transfer
- 2.6 – Implementing Secure System Calls
- 2.7 – Starting a New Process
- 2.8 – Implementing Upcalls
- 2.9 – Case Study: Booting an Operating Systems Kernel
- 2.10 – Case Study: Virtual Machines

Protection

An operating system must provide **protection** to prevent applications and users from corrupting applications and the OS itself.

- Reliability - prevents bugs in one program from crashing other programs or the OS.
- Security - prevents applications from modifying system state.
- Privacy - prevents unauthorized access to data.
- Fairness - limits access to system resources.

The Kernel

Protection is provided by the **operating system kernel**.

- The kernel has full access to machine hardware.
- All other applications run in a restricted environment.
- An application can request that the kernel perform operations on its behalf.
- Some applications must also provide protection from third-party extensions.

Processes

A **process** is the execution of an application with restricted rights.

- Any access of hardware devices or shared memory must go through the kernel.
- Requires hardware support to prevent applications from executing restricted operations.
- Protection must not impose too much overhead on applications.

The Process Abstraction (1)

When a program is executed, the OS allocates memory to store the following:

- The machine instructions that make up the compiled program.
- Space for statically allocated data.
- The **execution stack** to hold local variables and manage procedure calls.
- The **heap** for dynamically allocated data.

The Process Abstraction (2)

- A process encapsulates all these structures along with any additional system resources allocated to it.
- An OS may run multiple processes using the same program.
- A **process control block (PCB)** contains information used by the OS to keep track of a process.
- A process may need to keep track of multiple concurrent threads.

Dual-Mode Operation

- Modern processors support **dual-mode operation**.
- A bit in the processor status register determines the mode.
- In **user mode**, the processor checks to see if an instruction is permitted before executing it.
- In **kernel mode**, no protection checks are performed.

Support for Protection

To support protection, a processor must support the following:

- Privileged instructions that can only be executed in kernel mode.
- Memory protection that prevents access of memory outside of the region allocated to the process.
- Timer interrupts used by the kernel to regain control from a process.
- Safe transfer of control between user and kernel modes.

Privileged Instructions

A **privileged instruction** is an instruction that can only be executed in kernel mode.

Privileged instructions include the following:

- Changing privilege levels (other than via system calls.)
- Granting a process access to shared memory.
- Disabling and enabling interrupts.
- Creating and terminating other processes.
- Other operations involving concurrency, virtual memory, and file systems that we will discuss later on.

Memory Protection – Base and Bound

The simplest way to protect memory access is to use a **base and bound** approach.

- Base and bound registers store the first and last addresses accessible to the current running process.
- If an address being accessed is out of range, a processor exception is generated and the process is halted.
- The kernel runs without a base and bound.

Memory Protection – Drawbacks

The base and bound approach is easy to implement, but has several drawbacks.

- The amount of memory allocated is fixed, making it difficult to grow the stack and the heap.
- There is no support for sharing memory between processes.
- All addresses used in a program must be modified when the program is loaded.
- After a program is loaded it can’t be relocated without changing those addresses again, resulting in fragmentation of memory.

Memory Protection – Virtual Addresses

The problems with base and bound can be solved by using **virtual addresses**.

- Each process uses a contiguous range of virtual addresses starting at zero.
- The operating system translates the virtual addresses to physical addresses (e.g. by adding them to a base register.)
- Processes can be relocated without modifying any of the virtual addresses.

Timer Interrupts

Most computer systems include a hardware timer that can be used to periodically interrupt a processor.

- In a multiprocessor system each processor has its own timer.
- When a timer interrupt occurs the operating system switches to kernel mode.
- The kernel can then perform any needed operations before returning to user mode and resuming the current process.

User to Kernel Mode

A processor can switch from user to kernel mode for the following reasons:

- An external device generates an interrupt.
- A program executes an illegal instruction, generating a processor exception.
- A program performs a system call to explicitly request the kernel to perform a privileged operation.

Interrupts

An **interrupt** is an asynchronous signal to a processor that an external event has occurred.

- When an interrupt occurs, the processor saves the current execution state and jumps to an **interrupt handler**.
- A timer interrupt handler might check if a process is unresponsive or switch to a different process.
- Interrupts are generated when an I/O device produces input or completes an I/O operation.
- Alternatively the kernel can use a **polling** approach, repeatedly checking devices for new events.

Processor Exceptions

A **processor exception** is a hardware event caused by user program behavior.

- Occurs when accessing memory outside its own region.
- If a process is halted as a result, inter-processor interrupts may need to be sent to halt parallel processes.
- Generated by breakpoints inserted by a debugger.

System Calls

A **system call** is a procedure that can be called in user mode to request the kernel to perform a privileged operation.

- Required for performing network communication, file operations, and process creation.
- Appears like a normal procedure call to the user application.
- After completion, control is returned to the calling process at the instruction following the system call.

Kernel to User Mode

A processor must switch from kernel mode to user mode before an application can resume executing.

The following situations result in a switch back to user mode:

- A new process is loaded into memory and starts execution.
- An interrupt, processor exception, or system call has been processed and control is returned to the interrupted process.
- The kernel switches from the current running process to a different process.
- The kernel generates an **upcall**, which notifies a process of an event.

Implementing Safe Mode Transfer

- The switch between user and kernel mode requires a common sequence of instructions that must provide:
  - Limited entry into the kernel.
  - Atomic changes to the processor state.
  - Transparent, restartable execution.
- On an interrupt, the processor does the following
  1. Saves its current state to memory
  2. Temporarily disables interrupts
  3. Switches to kernel mode
  4. Executes the interrupt handler
  5. Restores the saved processor state

Interrupt Vector Table

How does the processor know what code to run when an interrupt occurs?

- A register contains the address of the **interrupt vector table**
- The table contains pointers to each of the **interrupt handlers**.
- When an interrupt occurs, the hardware selects the correct entry based on the source of the interrupt.
- The interrupt vector table must be stored in kernel memory to prevent processes from hijacking interrupts.

Interrupt Stack

- An **interrupt stack** is used to save the state of the current process and provide a context for the interrupt handler to execute in.
- When an interrupt occurs, the stack pointer is changed to the interrupt stack and the process's registers are pushed on the stack.
- When the interrupt handler completes, the registers are restored, including the stack pointer and program counter.
- Using a separate stack for the kernel provides reliability and security.
- On a multiprocessor each processor has its own interrupt stack.

Two Stacks per Process

- Most operating systems maintain an interrupt stack for every process running on the machine.
- If the process is running in user mode, the kernel stack is empty.
- If the process is running in kernel mode, the kernel stack contains the saved state of the process along with the current state of the handler.
- If the process is available to run, but waiting for it’s turn to run, the kernel stack contains the registers to be restored.
- If the process is waiting for an I/O event to complete, the kernel stack contains the suspended computation to be resumed when the I/O finished.

Interrupt Masking

- Interrupts arrive asynchronously, so they can not always be handled immediately.
- If an interrupt arrives when another interrupt is being handled, this could cause serious errors.
- Processors have instructions to **mask** (disable) and **unmask** (enable) interrupts, so they can be postponed until the kernel finishes an important operation.
- The hardware buffers interrupts, but space for buffering is limited, so interrupts should not be disabled for a long time.
- If the processor is in kernel mode with interrupts enabled, the state of each interrupt can be pushed onto the stack like a procedure call.

Saving and Restoring Registers

- Most processors provide special instructions for saving and restoring the state of a user-level process.
- The stack pointer, program counter, and status register must be pushed onto the interrupt stack before jumping to the handler code.
- The handler pushes any remaining registers onto the stack, handles the interrupt, then restores those registers.
- Finally, the interrupted process' stack pointer, program counter, and status register are restored.

Putting It All Together: x86 Mode Transfer (1)

On the x86 architecture a mode transfer performs the following operations:

1. Interrupts are disabled.
2. The current values of the stack pointer, program counter, and status register are saved in temporary registers.
3. The stack pointer register is set to the base of the kernel interrupt stack.
4. The three saved register values are pushed onto the stack.
5. An optional error code is pushed onto the stack.

Putting It All Together: x86 Mode Transfer (2)

6. The interrupt vector table is indexed by the interrupt type, and the program counter is set to the value at that index in the table.
7. The handler pushes the remaining registers, including the current stack pointer onto the stack.
8. When the handler completes, it restores the registers it saved and pops the error value off the stack.
9. The last instruction in the handler restores the stack pointer, program counter, and status register.
10. At this point interrupts are enabled and the interrupted process can continue where it left off.

Implementing Secure System Calls (1)

- A secure system call is implemented using a pair of **stubs.**
- The user program calls the **user stub** like a regular procedure.
- The user stub stores the code for the system call in a register and traps to the kernel.
- The hardware transfers control to the kernel, and invokes the system call handler.
- The handler acts as a **kernel stub**, copying and checking the arguments, then calling the kernel's implementation of the system call.

Implementing Secure System Calls (2)

- The kernel stub must do the following:
  1. Locate the system call arguments on the user stack.
  2. Copy parameters into kernel memory.
  3. Validate the parameters.
  4. Copy back any results of the system call to user memory.
- When the system call completes, it returns control to the handler.
- The handler returns to user level, giving control back to the user stub.
- The user stub returns to the caller.

Starting a New Process

To create a new process, the kernel must do the following:

1. Allocate and initialize the process control block.
2. Allocate memory for the process.
3. Copy the program from disk into memory.
4. Allocate a user-level stack.
5. Allocate a kernel-level stack.
6. Copy command-line arguments into user memory.
7. Transfer control to user mode.
8. Call a stub that calls the program's main function, then calls `exit`.

Implementing Upcalls

Applications can be notified of events by the kernel using **upcalls**.

- A user-level thread package can use a timer upcall to periodically switch threads.
- An upcall can notify a process that an asynchronous I/O operation has completed.
- Processes can use upcalls to communicate asynchronously.
- The kernel can use an upcall to notify a process of an exception it wants to handle.
- A process can use upcalls to adapt to a change in resource allocation.

UNIX Signals

UNIX signals are an example of kernel support for upcalls.

- The kernel defines a limited number of signals a process can receive.
- Each process defines its own handlers for each signal type.
- Signal handlers can be run on the normal execution stack or a separate signal stack.
- UNIX defers signals when the corresponding signal handler is in progress until that handler has returned to the kernel.
- The kernel copies saved state to the signal stack before executing the handler, and restores it after the handler returns.

Case Study: Booting an OS Kernel

When a computer boots, it uses the following steps to load the kernel:

1. The computer loads the basic input/output system (BIOS) from read-only memory (ROM).
2. The BIOS reads the bootloader from the boot disk into memory, and makes sure it has not been corrupted by a virus.
3. The bootloader loads the kernel into memory and jumps to it.
4. The kernel initializes its data structures and starts an initial process to start the system services.

Case Study: Virtual Machines (1)

A virtual machine starts a **guest OS** running on the **host OS** as follows:

1. The host loads the guest bootloader and starts it running.
2. The guest bootloader loads the guest kernel and starts it running.
3. The guest kernel initializes its interrupt vector table.
4. The guest kernel loads a process into guest memory.
5. To start the process, the guest traps into the host kernel.
6. The host kernel checks the validity of the mode transfer, then restores the critical registers of the process, and starts the process running.

Case Study: Virtual Machines (2)

When the guest process makes a system call, it traps to the host kernel, which does the following:

1. The host kernel saves the critical registers on the interrupt stack of the guest OS.
2. The host kernel transfers control to the guest kernel, which runs the interrupt handler in user mode.
3. The guest kernel performs the system call.
4. When the guest kernel returns from the system call, it traps into the host kernel.
5. The host kernel then restores the state of the process and resumes it in user mode.